מדריך לימוד עצמי — ביטים, ייצוג מספרים ופעולות ביטיות ב־C

המדריך מלווה את משימת הבונוס — הבייט האחרון. מטרתו לתת **נקודת פתיחה ומפת נושאים ללימוד עצמי** למי שלא למד מערכות ספרתיות. הוא אינו קורס מלא, ולא כל מה שצריך לדעת נמצא כאן. אם נושא מסוים אינו ברור, השלימו אותו באמצעות תיעוד השפה, חומרי לימוד ותרגול נוסף לפני המשך המשימה.

איך ללמוד: קראו פרק, הריצו את הדוגמאות, פתרו את שאלות התרגול בלי להעתיק את הקוד, ורק אחר כך עברו לשלב המתאים ב־`README.md`. מומלץ להשתמש ב־`-Wall -Wextra -Wpedantic -Werror` כדי לזהות טעויות מוקדם.

מפת הנושאים

פרק	מה לומדים	משימה
1	ביט, בית וטווחי ערכים	A
2	המרות בין בסיסים 2, 10 ו־16	A
3	מספרים שליליים ומשלים ל־2	B, E
4	`signed`, `unsigned`, גלישה והמרות	B, E
5	האופרטורים הביטיים ב־C	C, D
6	מסכות ביטים ושדות דחוסים	C, E
7	ASCII ו־XOR	D, F
8	קבצים בינאריים ו־Endianness	E, F

1. ביטים, בתים וגודל של מספר

ביט יכול להכיל 0 או 1. בעזרת `n` ביטים אפשר לייצג `2^n` תבניות שונות. לדוגמה, 8 ביטים מייצגים 256 תבניות, מ־`00000000` עד `11111111`.

בית (byte) הוא יחידת האחסון הבסיסית ב־C, ומספר הביטים בו מוגדר באמצעות `CHAR_BIT` מתוך `<limits.h>`. במשימת הבונוס פורמט הראיות מגדיר בתים בני **8 ביטים**. ב־C אפשר להשתמש ב־`uint8_t`, ב־`uint16_t` וב־`uint32_t` מתוך `<stdint.h>` כשרוצים רוחב מפורש.

תרגול: כמה ערכים שונים אפשר לייצג ב־4 ביטים? ומה הערך הלא חתום הגדול ביותר ב־12 ביטים?

2. בינארי, עשרוני והקסדצימלי

בסיס הספירה קובע אילו ספרות מותרות ומה המשקל של כל מיקום. בבינארי משתמשים ב־0 ו־1, בעשרוני ב־0–9, ובהקסדצימלי ב־0–9 וב־A–F. כל ספרה הקסדצימלית מייצגת בדיוק **ארבעה ביטים**.

למשל, הערך העשרוני `75` הוא `01001011` בבינארי ו־`0x4B` בהקסדצימלי: `75 = 11 + 16 × 4`. אלו דרכי כתיבה שונות של אותו ערך.

```c
unsigned a = 75;
unsigned b = 0x4B;          // אותו ערך
printf("%u %X\n", a, b);    // 75 4B
```

ב־**C11** אפשר לכתוב קבוע הקסדצימלי כמו `0x4B`, אך הקידומת `0b` לקבועים בינאריים אינה חלק מתקן C11 (גם אם מהדר מסוים תומך בה כהרחבה). לעומת זאת, מחרוזת כמו `"0b1011"` היא סתם טקסט, ואפשר לכתוב פונקציה שמפרשת אותה בעצמכם.

תרגול: המירו `42` לבינארי ולהקסדצימלי; המירו `0xAF` לעשרוני. כתבו לולאה שמחשבת ערך ממחרוזת ספרות בבסיס נתון.

3. מספרים שליליים — משלים ל־2

איך מייצגים `-5` בשמונה ביטים? בייצוג **משלים ל־2**, כותבים `5` בבינארי, הופכים את כל הביטים ומוסיפים 1:

```
+5:             00000101
היפוך הביטים:  11111010
הוספת 1:       11111011
```

הביטים `11111011` מייצגים `251` אם מפרשים אותם כ־8 ביטים **ללא סימן**, אך `-5` אם מפרשים אותם כ־8 ביטים **עם סימן** במשלים ל־2. בטווח של 8 ביטים, המספרים החתומים הם מ־`-128` עד `127`.

דרך פשוטה לפענח דפוס של `n` ביטים: אם הביט העליון הוא 0, הערך חיובי. אם הוא 1, אפשר לחשב `raw - 2^n`. לדוגמה: `251 - 256 = -5`.

תרגול: כתבו את `-15` במשלים ל־2 עם 8 ביטים. מהו הערך החתום של `0x80` ברוחב 8 ביטים? למה אין אותו מספר ערכים חיוביים ושליליים?

4. `unsigned`, `signed`, גלישה והמרות

טיפוס **לא חתום** (`unsigned`) משתמש בכל תבניות הביטים למספרים שאינם שליליים. טיפוס **חתום** (`signed`) כולל גם ערכים שליליים. אותו דפוס ביטים עשוי לקבל פירוש מספרי שונה בהתאם לרוחב ולסוג הייצוג.

ב־C צריך להבחין בין התנהגויות שונות:

- חישוב בטיפוס לא חתום של `n` ביטים מתבצע מודולו `2^n`: אחרי הערך המרבי חוזרים ל־0.
- **גלישה בחישוב של מספר שלם חתום היא התנהגות בלתי מוגדרת ב־C11.** אין להסתמך עליה כדי לבצע פענוח במשלים ל־2.
- המרה מערך לא חתום לערך חתום שאינו יכול להכיל אותו היא תלוית מימוש ב־C11. לכן בשלב B עדיף לחשב את הערך החתום באופן מפורש בטיפוס רחב מספיק.
- פעולות על `uint8_t` עשויות לעבור *integer promotions* לטיפוס `int`. חשוב להבין באיזה טיפוס מתבצע הביטוי בפועל.

תרגול: מה הטווח של מספר לא חתום בן 16 ביטים? מה קורה כאשר מוסיפים 1 ל־`UINT8_MAX` ואז שומרים את התוצאה ב־`uint8_t`? מה הסכנה בהשוואה בין `int` שלילי ל־`unsigned int`?

5. אופרטורים ביטיים ב־C

אלה האופרטורים החשובים:

אופרטור	פעולה	דוגמה
`&`	AND ביטי — 1 רק כאשר שני הביטים הם 1	`0xA & 0xC` נותן `0x8`
`|`	OR ביטי — 1 אם לפחות אחד הביטים הוא 1	`0xA | 0xC` נותן `0xE`
`^`	XOR ביטי — 1 כאשר הביטים שונים	`0xA ^ 0xC` נותן `0x6`
`~`	היפוך ביטים	הופך כל 0 ל־1 ולהפך
`>>`	הזזה שמאלה	`3u << 2` נותן `12`
`<<`	הזזה ימינה	`12u >> 2` נותן `3`

אין לבלבל בין `&` ו־`|` הביטיים לבין `&&` ו־`||` הלוגיים. אופרטורים לוגיים מחזירים אמת/שקר ומקצרים חישוב לפי הצורך; אופרטורים ביטיים עובדים על כל ביט של הערכים.

```
uint8_t flags = 0x05;       // 00000101
unsigned is_set = (flags & 0x04u) != 0u;
```

זהירות: אל תזיזו מספר שלם שלילי שמאלה; אל תבצעו הזזה במספר ביטים שלילי או במספר הגדול או שווה לרוחב הטיפוס שקיבל את ה־promotions. משמעות הזזה ימינה של מספר חתום שלילי תלויה במימוש ב־C11. במשימה נעדיף פעולות על ערכים לא חתומים.

תרגול: חשבו ביד את `0x3C ^ 0x0F`; מה ההבדל בין `5 & 2` לבין `5 && 2`?

6. מסכות ביטים וחילוץ שדות

מסכה היא מספר שבו הביטים שבחרנו הם 1. בעזרתה ניתן לבדוק או לשנות חלק מהביטים, בלי להשפיע על האחרים.

```
uint16_t word = 0xAB34;
unsigned top = (word >> 12) & 0x0Fu;
unsigned middle = (word >> 8) & 0x0Fu;
unsigned low = word & 0xFFu;
// top=0xA, middle=0xB, low=0x34
```

דפוסים שימושיים נוספים: `flags |= mask` מדליק את הביטים במסכה, `flags &= ~mask` מכבה אותם, ו־`flags ^= mask` הופך אותם. אם עובדים עם טיפוסים קטנים, זכרו שהאופרטור `~` פועל לאחר קידום טיפוסים; אפשר למסך שוב את התוצאה לרוחב הרצוי.

תרגול: חילצו את שמונת הביטים הנמוכים מתוך `0xBEEF`. כתבו ביטוי שבודק אם ביט מספר 5 דלוק. כתבו ביטוי שמכבה אותו מבלי לשנות ביטים אחרים.

7. בתים, ASCII ו־XOR

מחשב שומר תווים גם באמצעות ערכים מספריים. למשל, ב־ASCII האות `A` היא 65 (`0x41`) והאות `K` היא 75 (`0x4B`). מחרוזת C מסתיימת בבית שערכו `0`, המסומן `\0`.

פעולת XOR שימושית לתרגול: אם `encoded = original ^ key`, אז `original = encoded ^ key`, מפני ש־`k == x ^ (^ k)`. זוהי **אינה שיטת הצפנה בטוחה בעולם האמיתי**, אך היא ממחישה כיצד עובדים על בתים.

```
uint8_t original = 0x41;    // 'A'
uint8_t key = 0x5A;
uint8_t encoded = original ^ key;
uint8_t decoded = encoded ^ key; // חוזר ל-'A'
```

תרגול: מה יתקבל אם מפעילים פעמיים XOR עם אותו מפתח? מדוע `\0` חשוב בעת בניית מחרוזת? למה אי אפשר להניח שכל בית שפוענח מייצג תו שאפשר להדפיס?

8. קבצים בינאריים, `fread` ו־Endianness

בקובץ טקסט המספר `42` נשמר בדרך כלל כשני תווי ASCII (`'4'`, `'2'`). בקובץ בינארי הוא עשוי להישמר כבית יחיד שערכו 42, או כחלק משדה רחב יותר. כדי לפרש אותו צריך להכיר את **פורמט הקובץ**.

כאשר שומרים מספר רב־בתי, יש חשיבות לסדר הבתים:

- **Little-Endian:** הבית הפחות משמעותי ראשון. `0x1234` נשמר כ־`34 12`.
- **Big-Endian:** הבית המשמעותי ביותר ראשון. `0x1234` נשמר כ־`12 34`.

```
uint8_t raw[2] = {0x34, 0x12};
uint16_t value = (uint16_t)((uint16_t)raw[0] | ((uint16_t)raw[1] << 8));
// value == 0x1234
```

קוראים בתים מקובץ באמצעות `fread` ובודקים את מספר הבתים שנקראו. בהגיעכם לסוף הקובץ, הבדילו בין סוף תקין לבין שגיאת קריאה או רשומה שנקטעה באמצע.

```
uint8_t bytes[10];
size_t n = fread(bytes, 1, sizeof(bytes), file);
if (n != sizeof(bytes)) {
    // יש לבדוק אם זה סוף קובץ תקין, שגיאה או רשומה חלקית
}
```

אין לקרוא רצף בתים ישירות לתוך `struct` ולהניח שהשדות יתאימו: המבנה בזיכרון עשוי לכלול ריפוד (padding), וסדר הבתים של המחשב לא בהכרח תואם לפורמט הקובץ.

תרגול: כיצד מפענחים מהבתים `12 34 56 78` מספר בן 32 ביטים ב־Little-Endian? מה יקרה אם נפרש אותם בטעות כ־Big-Endian?

מה כדאי להשלים מעבר למדריך?

נושאי המשך מומלצים: *integer promotions*, המרות בין טיפוסים, ייצוג מספרים שליליים, יישור וריפוד מבנים (`padding` ו־`alignment`), והשפעת ייצוג הנתונים על קבצים ופרוטוקולים. אלה אינם בהכרח דרישות למימוש כל שלב בבונוס, אך הם מועילים להבנת תכנות מערכות ב־C.

כלל עבודה: כשמשהו אינו ברור, נסו קודם דוגמה קצרה משלכם, כתבו מה אתם מצפים שיקרה, הריצו והשוו. הבנה של הפער בין הציפייה לתוצאה היא חלק מרכזי מהתרגול.